INTRODUCTION:-

Digital Signal Processing (DSP) involves the analysis, modification, and synthesis of signals using digital techniques. Circular convolution is a fundamental DSP operation used to study linear time-invariant systems with periodic signals. Unlike linear convolution, circular convolution treats signals as periodic and wraps around when computing the output.

This case study demonstrates circular convolution using MATLAB, visualizes the input, impulse response, and output, and interprets the signal behavior.

Steps in MATLAB Code:

1. Define the input sequence x(n) and impulse response h(n).
2. Specify the convolution length N.
3. Perform circular convolution using MATLAB's cconv function.
4. Display sequences in the command window.
5. Visualize input, impulse response, and output on a polar plot.

Theory:-

Circular convolution is the process of combining two finite-length sequences in a way that the sequences are considered periodic with a fixed period N. Mathematically, for sequences x(n) and h(n) of length N, the circular convolution y(n) is given by:

$$\boldsymbol{y(n) = \sum_{m=0}^{N-1} x(m) \cdot h((n-m) \mathbf{mod}\ N)}$$

Key Points:

Circular convolution assumes the sequences are periodic and wraps the indices modulo N.

It is widely used in DSP for efficient computation using the Discrete Fourier Transform (DFT).

Circular convolution is different from linear convolution; linear convolution can be obtained by zero-padding sequences to length N.

Properties of Circular Convolution:

Commutativity: $x(n) \circledast h(n) = h(n) \circledast x(n)$

Associativity: $x(n) \circledast [h(n) \circledast g(n)] = [x(n) \circledast h(n)] \circledast g(n)$

Distributivity: $x(n) \circledast [h(n) + g(n)] = x(n) \circledast h(n) + x(n) \circledast g(n)$

Applications:

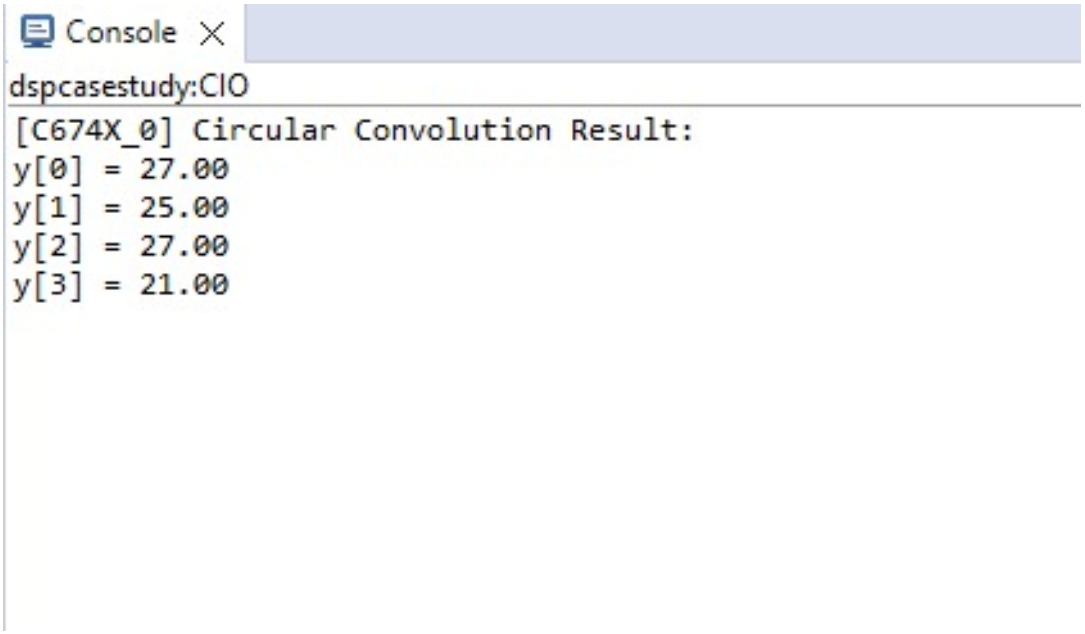
- Used in fast convolution algorithms for filtering operations.
- Essential in circular buffering for real-time signal processing.
- Plays a role in implementing efficient convolution using FFT (Fast Fourier Transform).

Relationship with DFT: Circular convolution in the time domain corresponds to element-wise multiplication in the frequency domain using the DFT: $\boldsymbol{y(n) = IDFT\{DFT[x(n)] \cdot DFT[h(n)]\}}$

This case study demonstrates circular convolution using MATLAB, visualizes the input, impulse response, and output, and interprets the signal behavior.

Code:-

```
#include <stdio.h>
#define N 4  // Length of the sequences
float x[N] = {1, 3, 2, 4};
float h[N] = {1, 2, 3, 4};
float y[N];
void circularConvolution(float *x, float *h, float *y, int n);
void main(void)
{
    int i;
    circularConvolution(x, h, y, N);
    printf("Circular Convolution Result:\n");
    for (i = 0; i < N; i++)
    {
        printf("y[%d] = %.2f\n", i, y[i]);
```

```
    }
    while(1); // keep running
}
void circularConvolution(float *x, float *h, float *y, int n)
{
    int i, j, k;
    for (i = 0; i < n; i++)
    {
        y[i] = 0;
        for (j = 0; j < n; j++)
        {
            k = (i - j);
            if (k < 0)
                k += n;
            y[i] += x[j] * h[k];
        }
    }
}
```

Output:-